A Multi-Agent Reflexion Framework with Two-Stage RAG

Tanay Kinariwala, Mahir Shah, Hetanshi Shah , Kailas K. Devadkar
Department of Computer Engineering, Sardar Patel Institute of Technology, Mumbai, India
Emails: {tanay.kinariwala23, amit.mahir23, hetanshi.shah23, kailas_devadkar}@spit.ac.in

Abstract Large Language Models (LLMs) enable scalable automated grading but often suffer from hallucinated feedback, fabricated evidence, and poor adherence to grading rubrics. To address these issues, we propose an automated paper-checking system that integrates a Two-Stage Retrieval-Augmented Generation (RAG) pipeline with a Multi-Agent Reflexion framework. The system processes mixed-media submissions using a hybrid native-OCR extraction module and grounds evaluations in verified course materials via bi-encoder retrieval followed by cross-encoder re-ranking. Grading is performed through an Actor–Critic loop, where an Actor generates rubric-based evaluations and a Critic audits them for factual consistency and unsupported claims. Failed evaluations are iteratively refined through a self-reflection step. Experimental results show reduced hallucinations and improved rubric adherence compared to zero-shot LLM baselines, while producing interpretable, evidence-backed feedback suitable for academic assessment.

***Index Terms*—Automated Grading, Large Language Models (LLMs), Multi-Agent Systems, Retrieval-Augmented Generation (RAG), Educational Technology, Fact Verification, Optical Character Recognition (OCR), Semantic Search.**

I. Introduction

The rapid advancement of Large Language Models (LLMs) has catalyzed significant interest in automated educational assessment, offering the potential to scale personalized feedback and reduce the administrative burden on educators. Traditional automated grading systems have historically relied on rigid keyword matching or regular expressions, which fail to capture the semantic nuances of student responses. While modern generative AI approaches address this by leveraging deep contextual understanding, deploying zero-shot or minimally prompted LLMs in high-stakes grading scenarios introduces critical vulnerabilities. Chief among these are the propensity for models to hallucinate factual information, fabricate supporting quotes from student texts, and deviate from strictly defined grading rubrics.

To bridge the gap between generative flexibility and rigorous academic assessment, we propose a novel, multi-agent automated paper-checking system designed to enforce factual grounding and strict rubric adherence. Our approach fundamentally departs from standard single-pass LLM wrappers by integrating a localized, verifiable knowledge base with a self-correcting agentic architecture.

To address the challenge of ingesting messy, mixed-media student submissions, our system implements a hybrid data extraction pipeline. It dynamically routes document pages through native PDF text extraction or optical character recognition (OCR) based on text density, ensuring high-fidelity digitization of both typed and scanned handwritten responses.

Crucially, to prevent the LLM from accepting plausible but factually incorrect student answers, we introduce a Two-Stage Retrieval-Augmented Generation (RAG) pipeline. This module grounds the grading process in authoritative course materials (e.g., digitized textbooks). By utilizing a fast bi-encoder for candidate chunk retrieval followed by a highly accurate cross-encoder for semantic re-ranking, the system provides the grading agents with precise ”truth contexts”.

The core evaluation is orchestrated through a Multi-Agent Reflexion Loop consisting of an ”Actor” and a ”Critic”. The Actor generates a preliminary grade based on the student’s text, the instructor’s rubric, and the retrieved truth context. Subsequently, the Critic acts as an independent auditor, verifying that the Actor did not ignore factual contradictions and that all cited evidence quotes are genuine. Evaluations that fail this audit trigger a reflexive regeneration, forcing the system to correct its hallucinations before finalizing the score.

The primary contributions of this paper are threefold:

1) **Hybrid Ingestion for Academic Submissions:** A robust PDF processing module combining PyMuPDF and PyTesseract to handle variable-quality student document scans seamlessly.
2) **Fact-Grounded RAG for Assessment:** The application of a dual-encoder RAG architecture to anchor LLM grading decisions in verified textbook knowledge, effectively mitigating domain-specific hallucinations.
3) **Actor-Critic Reflexion Framework:** A multi-agent evaluation loop that audits proposed grades for rubric compliance and factual integrity, providing a self-correcting mechanism for automated scoring.

The remainder of this paper is organized as follows: Section II reviews related work in LLM-based assessment and RAG systems. Section III details the proposed system architecture, including the extraction, retrieval, and multi-agent grading components. Section IV presents our experimental methodology and evaluation metrics. Section V discusses the results and system performance, and Section VI concludes the paper with directions for future research.

II. RELATED WORK

The evolution of automated educational assessment has progressed from early rule-based systems to modern AI-driven frameworks. This section reviews prior work across four major directions: traditional NLP and machine learning-based grading systems, Large Language Models (LLMs) in educational assessment, Retrieval-Augmented Generation (RAG) for factual grounding, and advanced prompting and multi-agent reasoning frameworks.

A. Traditional Machine Learning and NLP-based Assessment

Early automated grading systems relied on keyword matching, limiting their ability to capture semantic meaning. To overcome this limitation, Rokade *et al.* [1] proposed an NLP-based grading framework incorporating ontology-driven synonym expansion and contextual word matching to move beyond strict keyword dependence. Geigle *et al.* [2] framed automated grading of complex assignments as a supervised learning and ordinal regression task, demonstrating that active learning with human-in-the-loop strategies can significantly reduce manual grading effort while maintaining reliability.

Subsequent work adopted machine learning architectures. Xu [3] evaluated multiple ML classifiers including Naïve Bayes, Support Vector Machines, and Random Forests for grading written and programming assignments, reporting improved robustness over rule-based systems. Comprehensive surveys by Kumar *et al.* [4] and Wang *et al.* [5] further document how educational data mining and predictive analytics techniques have enhanced performance evaluation and academic outcome prediction.

Beyond objective assessments, automated grading has expanded to subjective and collaborative dimensions.

B. Large Language Models in Educational Assessment

The emergence of Large Language Models (LLMs) such as GPT-3 [9] and GPT-4 [10] has significantly advanced automated assessment. These models demonstrated unprecedented few-shot and zero-shot reasoning capabilities, making them attractive candidates for automated assessment tasks.

Xu *et al.* [11] provide a comprehensive survey of LLM applications in education, highlighting their role in tutoring, feedback, and scalable evaluation.

Empirical evidence supports LLM-based grading potential. Uyar and Büyükahıska [14] evaluated ChatGPT as an Automated Essay Scoring (AES) tool and reported strong correlation with human raters under standardized rubrics (e.g., IELTS), though notable statistical discrepancies remained. Despite scalability and cost-reduction benefits, Wang *et al.* [5] caution against inherent risks, including bias, limited explainability, and hallucinated feedback limitations that constrain standalone LLM deployment in high-stakes evaluation.

C. Grounding Evaluations with Retrieval-Augmented Generation (RAG)

To improve factual grounding in LLM outputs, researchers have proposed Retrieval-Augmented Generation (RAG). Lewis *et al.* [15] introduced RAG as a hybrid framework combining parametric knowledge within LLMs with non-parametric memory retrieved from dense vector indices. This approach enables models to ground their responses in external evidence.

Izacard and Grave [16] demonstrated that generative models significantly improve factual accuracy in open-domain question answering when augmented with retrieved passages. In educational assessment, RAG provides a mechanism to anchor grading decisions to verified instructional materials, rubrics, and course-specific references, thereby improving transparency and reliability.

D. Advanced Prompting and Multi-Agent Reflexion

While RAG improves grounding, complex tasks require structured reasoning. Wei *et al.* [17] introduced Chain-of-Thought (CoT) prompting, demonstrating that explicitly generating intermediate reasoning steps significantly improves logical and arithmetic problem-solving performance in LLMs.

Multi-agent frameworks like AutoGen [18] enable collaboration among specialized agents, while Reflexion [19] adds memory and self-reflection to iteratively correct errors without retraining.

E. Summary and Research Gap

Existing grading methods span NLP similarity models [7] to LLM-based scoring [14], but zero-shot LLMs suffer from hallucinations, rubric drift, and inconsistency. Advanced approaches like RAG [15] and Reflexion-based multi-agent frameworks [19] improve reasoning and knowledge use, yet are underexplored in academic evaluation.

Therefore, a research gap exists at the intersection of semantically grounded grading and structured multi-agent reasoning. The proposed framework addresses this gap by integrating a two-stage RAG pipeline with an Actor-Critic multi-agent loop, enabling automated grading that is semantically aligned, factually grounded, and iteratively self-correcting.

III. METHODOLOGY

A. System Overview

This study proposes an automated grading framework that evaluates student responses against a trusted knowledge base using a retrieval-grounded LLM pipeline. The system architecture consists of four sequential stages: hybrid document parsing, two-stage dense retrieval with cross-encoder re-ranking, a multi-agent Actor–Critic evaluation loop, and structured output generation.

Student submissions in PDF format are first converted to machine-readable text using a hybrid native/OCR extraction strategy. The extracted response is then grounded against authoritative course material via bi-encoder retrieval followed by cross-encoder re-ranking, yielding a high-confidence truth context for the grading agents. The Actor agent produces a rubric-aligned evaluation, which the Critic audits for factual consistency; evaluations failing this audit are revised by a Senior Grader using the critique feedback. The system outputs a structured JSON object containing scores, evidence

quotes, reasoning traces, and pipeline metadata to support transparency and reproducibility.

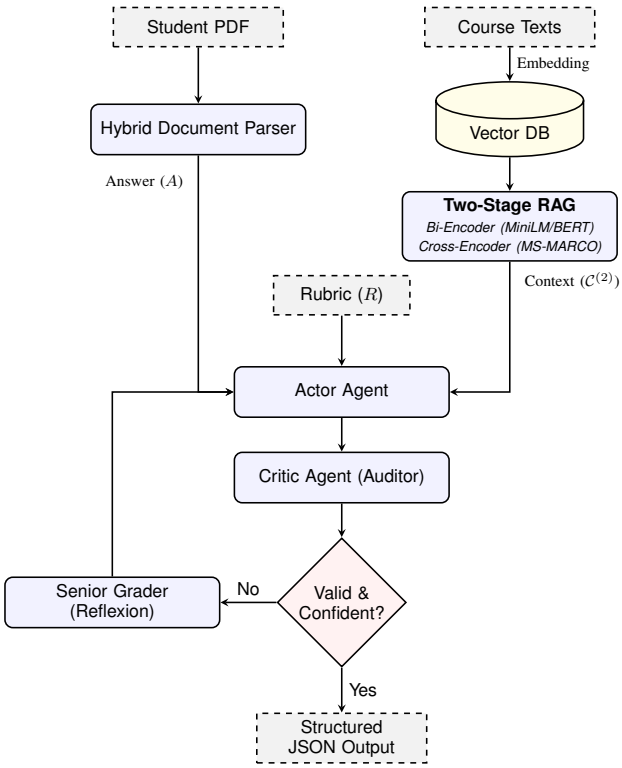

Fig. 1. End-to-end processing pipeline, showing vector embeddings, specific RAG models (MiniLM, MS-MARCO), and defined data flow (A, R, $\mathcal{C}^{(2)}$) into the Reflexion loop.

B. Hybrid Heuristic-Based Document Parsing

Student submissions often contain both digital and scanned content, making reliable text extraction challenging. The system employs a Hybrid Heuristic-Based Document Parsing strategy to maximize extraction accuracy while maintaining efficiency.

Let a document D consist of n pages:

$$D = \{p_1, p_2, \ldots, p_n\}$$

The pipeline begins by attempting native text extraction on each page p_i using the *PyMuPDF* library.This method is efficient and preserves structure for embedded text. Let $T(p_i)$ represent the string of text extracted natively from page p_i.

For scanned pages where parsing fails, the system applies a heuristic check.The system detects such cases by measuring the extracted text length $|T(p_i)|$.

We define a minimum character count threshold $\tau = 50$. If

$$|T(p_i)| < \tau$$

the system classifies the page as likely image-based or poorly encoded.

In these situations, the pipeline automatically switches to an Optical Character Recognition (OCR) process using the *Tesseract* engine through the *pytesseract* interface.The page is processed using OCR to recover text not captured by standard parsing.

The final text extraction function $E(p_i)$ for any given page is formally defined as a piecewise function:

$$E(p_i) = \begin{cases} T(p_i), & \text{if } |T(p_i)| \geq \tau \\ \text{OCR}(p_i), & \text{if } |T(p_i)| < \tau \end{cases}$$

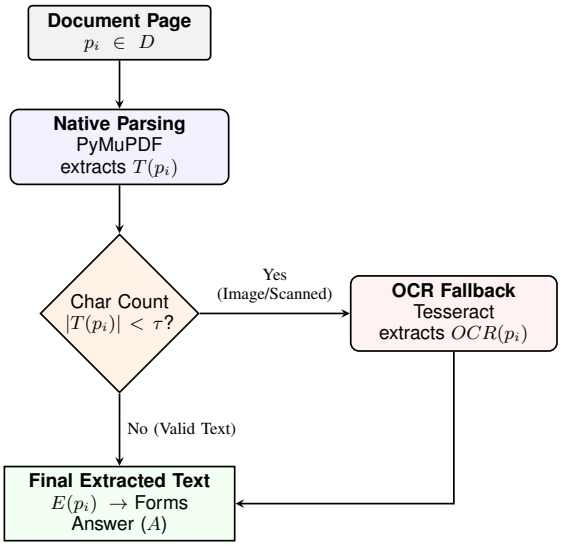

Fig. 2. The Hybrid Document Parsing strategy. Pages failing the minimum character threshold ($\tau = 50$) trigger the Tesseract OCR fallback to ensure high-fidelity text extraction (A).

C. Knowledge Base Construction

The knowledge base is constructed offline from trusted course materials such as textbooks, lecture notes, and instructor-provided reference documents, and serves as the factual foundation for the retrieval pipeline. Source documents are segmented using a sliding-window chunking strategy to preserve semantic continuity across chunk boundaries. Let a parsed source document S be represented as a sequence of m characters:

$$S = (c_1, c_2, \ldots, c_m)$$

We define chunk length $W = 1000$ and overlap $O = 200$ characters, giving stride $\Delta = W - O = 800$. The i-th chunk is:

$$C_i = (c_{i \cdot \Delta}, \ldots, c_{i \cdot \Delta + W - 1}), \quad i \in \{0, 1, 2, \ldots\}$$

Each chunk is then encoded into a dense vector representation using `all-MiniLM-L6-v2`, mapping each text segment to a d-dimensional semantic space:

$$f_{\text{emb}} : \mathcal{X} \rightarrow \mathbb{R}^d, \qquad v_i = f_{\text{emb}}(C_i)$$

The resulting vectors, along with their corresponding text content and metadata, are stored in a local indexed repository.

Formally, the knowledge base is defined as the set of all chunk-vector pairs:

$$\mathcal{KB} = \{(C_i, v_i, \mathcal{M}_i) \mid 0 \leq i \cdot \Delta \leq m - W\}$$

where $\mathcal{M}_i$ denotes the metadata associated with chunk C_i, such as source document name and page number. This offline indexing ensures that embeddings are computed once and reused across all grading evaluations.

D. Two-Stage Dense Retrieval with Cross-Encoder Re-Ranking

To ensure relevant contextual grounding, the system employs a two-stage dense retrieval pipeline that balances efficiency and semantic accuracy. Traditional single-stage retrieval methods often struggle to identify the most contextually appropriate passages, particularly when queries contain complex academic language or implicit concepts. This approach combines fast candidate retrieval with precise re-ranking.

In the first stage, a bi-encoder retrieval model is used to rapidly identify relevant knowledge segments from the knowledge base $\mathcal{KB}$. Let the evaluation query be denoted as q. The query q and knowledge chunks are encoded into dense vectors using the same embedding model f_{emb}:

$$v_q = f_{\text{emb}}(q)$$

Cosine similarity is then computed between the query vector v_q and all stored chunk embeddings $v_i \in \mathcal{KB}$ to produce an initial relevance ranking. The bi-encoder similarity score s_{bi} is formally defined as:

$$s_{\text{bi}}(q, C_i) = \frac{v_q \cdot v_i}{\|v_q\| \|v_i\|}$$

The system selects the top N candidate chunks (typically N = 20):

$$\mathcal{C}^{(1)} = \underset{C_i \in \mathcal{KB}}{\arg\max}^{(N)} s_{\text{bi}}(q, C_i)$$

While bi-encoder retrieval is efficient, it evaluates the query and documents independently, limiting deeper contextual understanding. To address this limitation, the second stage applies a cross-encoder re-ranking model that jointly processes the query q and each candidate passage $C \in \mathcal{C}^{(1)}$.This model performs joint semantic evaluation and assigns a refined score s_{cross}.

The refined score is computed as:

$$s_{\text{cross}}(q, C) = f_{\text{cross}}(q, C)$$

The passages are then re-ordered according to these refined scores, and the top K segments (default value $K = 5$) are selected as the final Truth Context $\mathcal{C}^{(2)}$ used during grading:

$$\mathcal{C}^{(2)} = \underset{C \in \mathcal{C}^{(1)}}{\arg\max}^{(K)} s_{\text{cross}}(q, C)$$

This two-stage design improves contextual relevance while maintaining efficiency.

E. Multi-Agent LLM Evaluation Architecture

To improve the reliability and transparency of automated grading, the proposed system employs a multi-agent large language model (LLM) architecture in which multiple specialized agents collaborate to produce and validate grading decisions. Unlike traditional single-model evaluation systems, this architecture distributes responsibilities across distinct roles, allowing independent verification and reducing the likelihood of unsupported conclusions.

The evaluation process begins with an **Actor** agent, which functions as the primary academic grader. Let the student's extracted response be denoted as A, the defined grading rubric as R, and the retrieved Truth Context generated by the retrieval pipeline as $\mathcal{C}^{(2)}$. We define the Actor agent as a generative function $\mathcal{M}_{\text{Actor}}$. Using these inputs, the Actor produces an initial assessment E_0:

$$E_0 = \langle m, r, e \rangle = \mathcal{M}_{\text{Actor}}(A, R, \mathcal{C}^{(2)})$$

where m represents the proposed numerical score, r is the explanatory reasoning trace, and e is the supporting evidence drawn from the retrieved material. The agent is explicitly instructed to prioritize factual consistency with the provided context and to penalize answers that contradict verified information.

Following the initial grading step, a second agent known as the **Critic** performs an independent audit of the Actor's output. The Critic examines whether the evaluation correctly reflects the rubric, verifies that the cited evidence actually exists within the retrieved context, and checks for logical inconsistencies or overlooked information.

Let the Critic agent be defined as a verification function:

$$\mathcal{M}_{\text{Critic}} : (\mathcal{X}, \mathcal{X}, \mathcal{X}, \mathcal{X}) \to (\{0, 1\}, [0, 1], \mathcal{X})$$

In addition to a textual critique t_c, the Critic produces a binary validity indicator $v \in \{0, 1\}$ and a continuous confidence score $c \in [0, 1]$ representing the reliability of the initial evaluation. The Critic output is defined as:

$$O_C = \langle v, c, t_c \rangle = \mathcal{M}_{\text{Critic}}(E_0, A, R, \mathcal{C}^{(2)})$$

This role-based architecture introduces separation between generation and verification, which improves robustness and helps detect hallucinated evidence or weak reasoning. By structuring the grading workflow as a collaborative interaction between specialized agents, the system encourages more consistent, transparent, and academically grounded evaluation outcomes. The interaction between these agents is further refined through an iterative critique and revision process described in the following section.

F. Reflexion-Based Critique Loop

Automated grading systems often produce incomplete or weakly justified assessments when relying on a single inference step. The proposed approach addresses this limitation and further improve grading reliability, the system incorporates

a **Reflexion-based critique loop** that enables iterative self-correction of the evaluation process.

After the initial assessment E_0 is generated by the Actor agent and audited by the Critic, the system evaluates O_C to determine whether the grading decision is sufficiently reliable. We define a minimum confidence threshold $\theta = 0.85$.

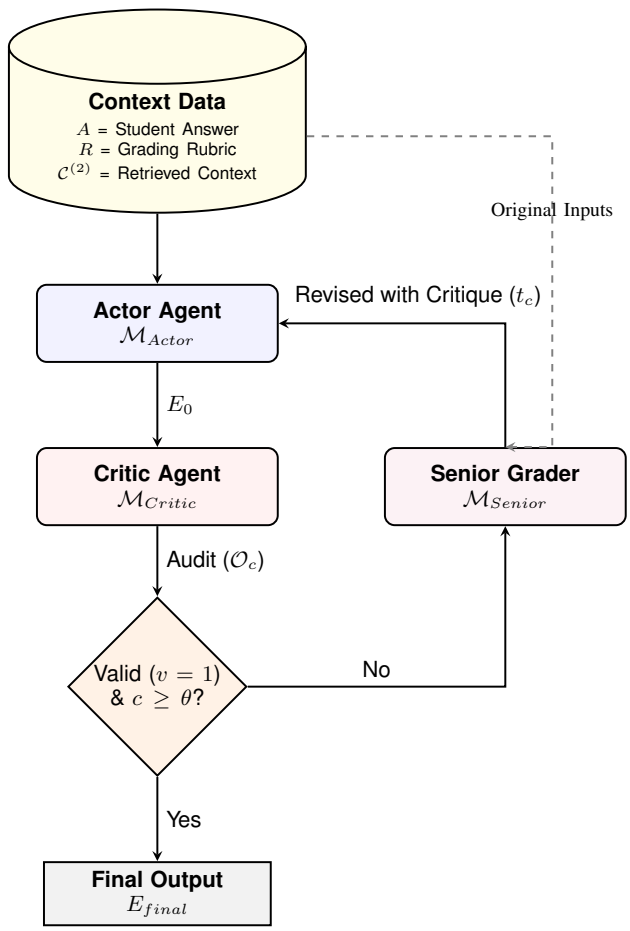

Fig. 3. The Multi-Agent Reflexion Loop. The context variables (A, R, $\mathcal{C}^{(2)}$) are routed to the Actor, while the Critic's audit triggers context-grounded revisions by the Senior Grader.

If the output is marked as invalid ($v = 0$) or if the confidence score falls below the predefined threshold ($c < \theta$), the system triggers a revision stage.

During this stage, a **Senior Grader** agent, denoted as the function $\mathcal{M}_{\text{Senior}}$, is invoked to regenerate the evaluation using the Critic's feedback as guidance. The Senior Grader receives the original inputs $(A, R, \mathcal{C}^{(2)})$ along with the initial evaluation E_0 and the critique notes t_c highlighting potential issues such as missing evidence, incorrect reasoning, or misalignment with the rubric.

By explicitly incorporating this feedback, the agent is able to revise the grading decision and produce a more accurate and justified assessment E_{revised}:

$$E_{\text{revised}} = \mathcal{M}_{\text{Senior}}(A, R, \mathcal{C}^{(2)}, E_0, O_C)$$

Formally, the final grading output E_{final} of the Reflexion loop is determined by the following piecewise function:

$$E_{\text{final}} = \begin{cases} E_0, & \text{if } v = 1 \text{ and } c \geq \theta \\ E_{\text{revised}}, & \text{otherwise} \end{cases}$$

This iterative critique mechanism encourages reflective reasoning within the multi-agent framework and reduces the likelihood of unsupported conclusions and therefore functions as a quality control layer, ensuring that the final grading output is consistent with both the retrieved knowledge context and the evaluation criteria defined by the rubric.

G. Evidence-Grounded Scoring

To ensure transparency, the framework adopts an evidence-grounded scoring mechanism that requires all decisions to be supported by retrieved context. The model is constrained to justify each decision using explicit references to the retrieved context.

During evaluation, the agent compares the student response with rubric criteria and retrieved context.Let the rubric R consist of k criteria:

$$R = \{r_1, r_2, \ldots, r_k\}$$

where each criterion r_j has a maximum assignable weight W_j.

For every rubric component r_j, the agent evaluates the conceptual alignment between the student's response A and the Truth Context $\mathcal{C}^{(2)}$ to evaluate conceptual alignment. We define an alignment function:

$$f_{\text{align}}(A, r_j, \mathcal{C}^{(2)}) \to \alpha_j$$

where $\alpha_j \in [0, 1]$ represents the degree of factual correctness and completeness. The awarded marks m_j for criterion r_j are therefore computed as:

$$m_j = \alpha_j \cdot W_j$$

This ensures marks reflect alignment and penalize missing or incorrect concepts.

For interpretability, the agent provides two explanations per criterion:

$$T_j = \langle m_j, \rho_j, \epsilon_j \rangle$$

where ρ_j represents the reasoning trace summarizing the reasoning behind the score, and $\epsilon_j \subseteq \mathcal{C}^{(2)}$ denotes an exact evidence quote extracted from the retrieved context that supports the evaluation.

The total score M for the response is computed as the summation over all rubric components:

$$M = \sum_{j=1}^{k} m_j$$

This ensures explainable and evidence-linked grading.

By enforcing explicit evidence usage, the scoring mechanism reduces the likelihood of hallucinated justifications and encourages fact-consistent assessments. This design also enables instructors and researchers to audit grading decisions, making the evaluation pipeline more trustworthy and suitable for deployment in educational settings.

H. Structured Output Schema

To ensure interoperability, transparency, and ease of downstream analysis, the proposed system produces all grading results using a structured output schema. Rather than generating free-form textual feedback alone, the evaluation pipeline formats its final output as a standardized JSON object. This design allows the grading results to be easily integrated with front-end dashboards, analytics tools, and academic management systems. Formally, the output object $\mathcal{O}$ is defined as a tuple of the grading evaluation array $\mathcal{G}$ and system-level metadata $\mathcal{M}$:

$$\mathcal{O} = \langle \mathcal{G}, \mathcal{M} \rangle$$

Let $Q = \{q_1, q_2, \ldots, q_n\}$ denote the set of evaluated questions. Each question $q_i \in Q$ is represented as an entry within the evaluation array $\mathcal{G}$. Each entry $\mathcal{G}_i \in \mathcal{G}$ contains the assigned marks m_i, a reasoning trace ρ_i explaining the grading decision, and a supporting evidence quote ϵ_i extracted from the retrieved knowledge context:

$$\mathcal{G}_i = \langle m_i, \rho_i, \epsilon_i \rangle, \qquad \mathcal{G} = [\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_n]$$

By enforcing this structure, the system ensures that every score is accompanied by a transparent justification grounded in verifiable source material. In addition to grading information, the metadata component $\mathcal{M}$ records system-level statistics including total processing latency T_{total}, document parsing statistics, and the extraction method applied to each page during ingestion. Capturing these details enables reproducibility and provides insight into the internal behavior of the pipeline.

I. System Metrics and Logging

To evaluate system performance, a metrics and logging module records key information throughout the pipeline. These measurements provide insight into system behavior, enable reproducibility, and support empirical analysis in research settings.

During processing, the framework tracks multiple categories of metrics, including document ingestion statistics, retrieval behavior, agent evaluation outcomes, and overall processing latency. For the document parsing stage, the system records which extraction method $E(p_i)$ was applied to each page (native parsing or OCR). To quantify document quality and the frequency with which fallback OCR processing is required, we define the OCR utilization rate μ_{OCR} for an n-page document using an indicator function $\mathbb{I}$:

$$\mu_{\text{OCR}} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}\big(|T(p_i)| < \tau\big)$$

The retrieval stage logs the number of candidate chunks retrieved (N), re-ranking operations (K), and the final context size (total characters or tokens) used during grading. Monitoring these values allows researchers to evaluate the efficiency and effectiveness of the two-stage retrieval strategy.

Similarly, the multi-agent evaluation module records audit outcomes v, confidence scores c, and whether the Reflexion revision process was triggered. For a set of evaluated questions Q, the Reflexion trigger rate $\mu_{\text{Reflexion}}$ across the evaluation is formally expressed as:

$$\mu_{\text{Reflexion}} = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \mathbb{I}\big(v_j = 0 \vee c_j < \theta\big)$$

The system measures total processing latency as $T_{\text{total}} = T_{\text{ingest}} + T_{\text{retrieve}} + T_{\text{evaluate}}$ to assess scalability and usability.

All metrics are packaged as $\mathcal{M} = \langle \mu_{\text{OCR}}, \mu_{\text{Reflexion}}, T_{\text{total}}, \ldots \rangle$ alongside results for analysis, benchmarking, and optimization.

J. End-to-End Pipeline

The complete grading workflow operates as a sequential pipeline that integrates document processing, retrieval, multi-agent evaluation, and structured reporting. The process begins when a student submission D is uploaded in PDF format and processed by the hybrid parsing module. Formally, the extracted student response A is the concatenation of all parsed pages:

$$A = \bigoplus_{i=1}^{n} E(p_i)$$

The system then constructs an evaluation query q by combining the extracted response with the grading rubric R:

$$q = A \oplus R$$

The query q is passed to the two-stage dense retrieval module, which searches the pre-constructed knowledge base $\mathcal{KB}$ to produce the high-confidence Truth Context $C^{(2)}$. The retrieved context, together with A and R, is then provided to the Actor–Critic evaluation loop. If the Critic's audit detects issues or produces a confidence score $c < \theta$, the Senior Grader is invoked to yield the final evaluation E_{final}. The results are then compiled into the structured output:

$$\mathcal{O} = \langle \mathcal{G}, \mathcal{M} \rangle$$

The entire end-to-end grading pipeline can therefore be summarized as a composite mapping:

$$\Phi : \mathcal{D} \times \mathcal{R} \to \mathcal{O}$$

IV. Results

The proposed framework was evaluated on a dataset of 53 student assignments from a Distributed Computing File Systems course. Each assignment was graded manually by the instructor using a predefined rubric, and these scores were treated as ground truth.

Let y_i denote the actual (instructor-assigned) marks and $\hat{y}_i$ denote the predicted marks generated by the system for $N = 53$ samples.

A. Evaluation Metrics

To quantify grading performance, we compute standard regression metrics:

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{1}$$

Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{2}$$

Pearson Correlation Coefficient (r):

$$r = \frac{\sum_{i=1}^{N}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^{N}(y_i - \bar{y})^2 \sum_{i=1}^{N}(\hat{y}_i - \bar{\hat{y}})^2}} \tag{3}$$

B. Quantitative Results

The proposed system achieves the following performance:

- MAE = 2.30
- RMSE = 2.37
- Correlation (r) = 0.9897

C. Discussion

The low MAE and RMSE indicate that the predicted scores deviate by only 2–3 marks on average (out of 100), demonstrating high grading precision.

These results validate the effectiveness of the proposed Two-Stage RAG and Multi-Agent Reflexion framework in producing accurate, reliable, and consistent grading outcomes.

V. Conclusion

This paper presented a reliable automated grading framework that integrates a Two-Stage Retrieval-Augmented Generation (RAG) pipeline with a Multi-Agent Reflexion architecture. By combining hybrid document parsing, retrieval-grounded evaluation, and an Actor–Critic verification loop, the system addresses key limitations of zero-shot LLM-based grading, particularly hallucinations and inconsistent rubric adherence.

Experimental results on 53 real-world assignments demonstrate strong alignment with instructor grading, achieving a low MAE of 2.30, RMSE of 2.37, and a high correlation of 0.9897. These findings validate that the proposed approach produces accurate, consistent, and interpretable grading outcomes.

Retrieval grounding ensures factual accuracy, while the Reflexion-based multi-agent loop enhances reliability through iterative self-correction for evidence-based academic evaluation.

Future work will focus on scaling the framework to larger datasets, extending support for diverse subject domains, and optimizing latency for real-time deployment in educational environments.

References

[1] A. Rokade, B. Patil, S. Rajani, S. Revandkar and R. Shedge, "Automated Grading System Using Natural Language Processing," 2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT), Coimbatore, India, 2018, pp. 1123-1127, doi: 10.1109/ICICCT.2018.8473170.

[2] C. Geigle, C. Zhai, and D. Ferguson, "An Exploration of Automated Grading of Complex Assignments," in *Third (2016) ACM Conference on Learning @ Scale (L@S '16)*.

[3] D. C. F. Xu, "Design and Implementation of Machine Learning Algorithms in Automatic Grading of Students' Assignments," *Journal of Electrical Systems*, vol. 20-3s, pp. 899–919, 2024.

[4] C. V. A. Kumar, A. K. Eemani, G. C. Kalluri, and G. Rudra, "A survey on automated student evaluation and analysis using machine learning," *World Journal of Advanced Research and Reviews*, vol. 21, no. 2, pp. 2547–2554, 2024.

[5] J. Wang, Y. Hussain, X. Zhang, C. Mao, and Z. Wu, "AI-Based Automated Grading Systems: Opportunities, Challenges, and Future Directions," *Business and Social Sciences Proceedings Review (GEEA)*, 2025.

[6] B. B. Tomić, A. D. Kijevčanin, Z. V. Ševarac, and J. M. Jovanović, "An AI-based Approach for Grading Students' Collaboration," *IEEE Transactions on Learning Technologies*, vol. 16, no. 3, pp. 292–305, 2023.

[7] A. Ayaan and K.-W. Ng, "Automated grading using natural language processing and semantic analysis," *MethodsX*, vol. 14, p. 103395, 2025.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin *et al.*, "Attention Is All You Need," in *31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.*.

[9] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," arXiv preprint arXiv:2005.14165, 2020. Available: https://arxiv.org/abs/2005.14165.

[10] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023. Available: https://arxiv.org/abs/2303.08774.

[11] H. Xu, W. Gan, Z. Qi, J. Wu, and P. S. Yu, "Large Language Models for Education: A Survey," arXiv preprint arXiv:2405.13001, 2024. Available: https://arxiv.org/abs/2405.13001.

[12] J. Gardner, M. O'Leary, and L. Yuan, "Artificial intelligence in educational assessment: 'Breakthrough? Or buncombe and ballyhoo?,'" *Journal of Computer Assisted Learning*, vol. 37, no. 5, pp. 1207–1216, 2021.

[13] I. Saputra, A. Kurniawan, M. Yanita, E. Y. Putri, and M. Mahniza, "The Evolution of Educational Assessment: How Artificial Intelligence is Shaping the Trends and Future of Learning Evaluation," *Indonesian Journal of Computer Science*, vol. 13, no. 6, 2024.

[14] A. C. Uyar and D. Büyükahıska, "Artificial intelligence as an automated essay scoring tool: A focus on ChatGPT," *International Journal of Assessment Tools in Education*, vol. 12, no. 1, pp. 20–32, 2025.

[15] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," arXiv preprint arXiv:2007.01282, 2020. Available: https://arxiv.org/abs/2007.01282.

[17] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[18] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint arXiv:2308.08155, 2023. Available: https://arxiv.org/abs/2308.08155.

[19] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, "Reflexion: an autonomous agent with dynamic memory and self-reflection," arXiv preprint arXiv:2303.11366, 2023. Available: https://arxiv.org/abs/2303.11366.